LAPORAN PRAKTIKUM PENGEMBANGAN APLIKASI BERBASIS WEB

Modul 2: STRUKTUR PROYEK DAN ROUTING DAN MEMBUAT APLIKASI WEB SEDERHANA DENGAN LARAVEL (TANPA DATABASE)



Universitas Telkom

Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S

NIM: 707012530010

Kelas: D4SIKC-49-04

Mata Kuliah: Pengembangan Aplikasi Berbasis Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan

Adapun tujuan yang ingin dicapai pada praktikum Modul 2 ini adalah sebagai berikut:

1. Memahami struktur direktori utama proyek Laravel serta peran folder app, routes, resources, dan public dalam penerapan pola arsitektur MVC.
2. Mampu menuliskan berbagai variasi routing pada Laravel, mulai dari routing sederhana, routing dengan parameter, parameter opsional, routing menuju view, pengiriman data ke view, hingga named route.
3. Mampu membangun aplikasi web sederhana bernama LaporBanjir tanpa database yang memiliki alur input, proses, dan tampil hasil menggunakan route, controller, dan view Blade.
4. Mampu mengintegrasikan CSS pada file view agar tampilan form dan halaman hasil pada setiap aplikasi lebih rapi dan mudah digunakan.
5. Mampu membuat seluruh route sesuai rancangan Tugas Akhir yang dapat diakses melalui browser, serta membangun aplikasi web sederhana tanpa database yang merepresentasikan salah satu proses pengolahan data pada rancangan Tugas Akhir, yaitu input, proses, dan tampilkan hasil data registrasi siswa.

2. Alat dan Bahan

Praktikum ini dikerjakan menggunakan satu buah laptop dengan sistem operasi Windows dan XAMPP sebagai lingkungan pengembangan web lokal. Perangkat lunak yang digunakan meliputi PHP versi 8.2.12 sebagai bahasa pemrograman sisi server, Composer versi 2.10.3 sebagai dependency manager, Laravel Framework versi 12 sebagai framework utama aplikasi, Visual Studio Code sebagai editor kode untuk menuliskan program, serta Google Chrome sebagai browser untuk menjalankan dan menguji aplikasi.

Proyek Laravel ditempatkan pada direktori D:\XAMPP\htdocs\pabw-1. Pada praktikum ini tidak ada penggunaan database untuk menyimpan data aplikasi, sehingga fokus pengerjaan berada pada route, controller, dan view.

3. Langkah-Langkah Praktikum

Langkah-langkah praktikum ini diuraikan sesuai tiga tugas pada modul, yaitu (1) membangun aplikasi pelaporan banjir LaporBanjir tanpa database, (2) mendefinisikan route sesuai rancangan Tugas Akhir, dan (3) membuat aplikasi sederhana input-proses-tampil untuk proses registrasi data siswa pada rancangan Tugas Akhir.

3.1 Membuat Proyek Laravel

Proyek Laravel dibuat pada direktori D:\XAMPP\htdocs\pabw-1. Struktur direktori utama yang dihasilkan antara lain folder app/ yang memuat controller pada app/Http/Controllers, folder routes/ yang memuat definisi rute pada file web.php, folder resources/views yang memuat file tampilan Blade, serta folder public/ yang menjadi satu-satunya folder yang dapat diakses langsung oleh browser, termasuk berkas CSS pada public/css dan berkas JavaScript pada public/js. Pada praktikum ini tidak ada penggunaan database, sehingga fokus pengerjaan berada pada route, controller, dan view.

3.2 Mendefinisikan Route

Route didefinisikan pada file routes/web.php. Selain route latihan materi seperti /home, /welcome, /hello, /user/{name}, /user/{name?}, /about, dan /profile/{name?} beserta named route home.page dan welcome.page, pada file yang sama juga didefinisikan tiga route untuk aplikasi LaporBanjir:

```
use App\Http\Controllers\LaporBanjirController;

Route::get('/laporbanjir', [LaporBanjirController::class, 'form'])->name('laporbanjir.form');
Route::post('/laporbanjir', [LaporBanjirController::class, 'proses'])->name('laporbanjir.proses');
Route::get('/laporbanjir/konfirmasi', [LaporBanjirController::class, 'konfirmasi'])->name('laporbanjir.konfirmasi');
```

Route GET /laporbanjir memanggil method form() untuk menampilkan halaman input, route POST /laporbanjir memanggil method proses() untuk mengolah data kiriman form, dan route GET /laporbanjir/konfirmasi memanggil method konfirmasi() untuk menampilkan ringkasan. Route POST harus digunakan karena form pada view memakai method POST.

3.3 Membuat Controller LaporBanjir

Controller dibuat melalui perintah artisan berikut, yang secara otomatis menghasilkan file app/Http/Controllers/LaporBanjirController.php:

```
php artisan make:controller LaporBanjirController
```

Method form() menampilkan view form pelaporan, method proses() memvalidasi data masukan lalu menghitung kategori genangan, dan method konfirmasi() menampilkan hasil olahan. Inti dari method proses() adalah sebagai berikut:

```
$validated = $request->validate([
    'nama' => 'required|string|max:100',
    'kecamatan' => 'required|string|max:100',
    'desa' => 'required|string|max:100',
    'tinggi_genangan' => 'required|numeric|min:0|max:1000',
]);

$tinggi = (float) $validated['tinggi_genangan'];

if ($tinggi >= 70) {
    $kategori = 'Banjir Tinggi';
} elseif ($tinggi >= 30) {
    $kategori = 'Genangan Sedang';
} else {
    $kategori = 'Genangan Rendah';
}
```

Kategori ditentukan dari tinggi genangan, yaitu 70 cm atau lebih termasuk Banjir Tinggi, 30 sampai 69 cm termasuk Genangan Sedang, dan di bawah 30 cm termasuk Genangan Rendah. Hasil olahan dikirim ke halaman konfirmasi melalui session flash dengan redirect()->route('laporbanjir.konfirmasi')->with('laporan', $laporan), sehingga data hanya bertahan selama satu pengalihan dan tidak disimpan ke database. Method konfirmasi() memeriksa keberadaan session 'laporan' dan mengembalikan pengguna ke halaman form jika session tidak ditemukan.

3.4 Membuat View Form Pelaporan LaporBanjir

View dibuat pada file resources/views/laporbanjir/form.blade.php. Halaman ini merupakan HTML utuh yang memuat CSS dan JavaScript eksternal melalui helper asset(), form diarahkan ke named route laporbanjir.proses dengan method POST dan wajib menyertakan directive @csrf agar request tidak ditolak oleh perlindungan CSRF Laravel. Setiap input diberi atribut required agar pengguna tidak mengirim laporan kosong. Inti dari file tersebut adalah sebagai berikut:

```
<link rel="stylesheet" href="{{ asset('css/laporbanjir.css') }}">

<form class="kartu form" id="form-lapor" action="{{ route('laporbanjir.proses') }}" method="POST">
   @csrf
   <div class="field">
      <label class="field__label" for="nama">Nama pelapor</label>
      <input class="field__input @error('nama') field__input--galat @enderror"
           type="text" id="nama" name="nama" value="{{ old('nama') }}" required>
      @error('nama')
         <p class="field__galat" role="alert">{{ $message }}</p>
      @enderror
   </div>
   ...
</form>
```

Selain menampilkan pesan galat validasi melalui directive @error, form ini juga menampilkan pratinjau kategori secara langsung. JavaScript pada public/js/laporbanjir.js membaca nilai tinggi genangan dan menampilkan perkiraan kategori pada elemen pratinjau tanpa mengirim data ke server.

3.5 Membuat View Konfirmasi LaporBanjir

View kedua dibuat pada file resources/views/laporbanjir/konfirmasi.blade.php sebagai halaman konfirmasi laporan. Data yang dikirim controller melalui session ditampilkan menggunakan sintaks Blade {{ $variabel }}. Halaman ini menampilkan nomor referensi, identitas pelapor, lokasi kejadian, waktu lapor, serta visualisasi tinggi genangan dalam bentuk skala dan badge kategori:

```
<dl class="ringkasan__daftar">
   <div class="ringkasan    baris">
      <dt>Nomor referensi</dt>
      <dd>{{ $laporan['referensi'] }}</dd>
   </div>
   ...
</dl>

<p class="badge {{ $kelasKategori }}">{{ $laporan['kategori'] }}</p>
```

Halaman konfirmasi juga memakai CSS eksternal yang sama dengan halaman form, sehingga kedua halaman terlihat sebagai satu kesatuan aplikasi. Sebuah tautan menuju route laporbanjir.form disediakan agar pengguna dapat membuat laporan baru.

3.6 Menjalankan dan Menguji Aplikasi LaporBanjir

Aplikasi dijalankan melalui development server Laravel dengan perintah php artisan serve, kemudian diakses pada alamat http://127.0.0.1:8000/laporbanjir. Pengguna mengisi nama pelapor, kecamatan, desa, dan tinggi genangan air, lalu menekan tombol Kirim Laporan. Data dikirim melalui method POST ke route /laporbanjir, diproses oleh controller, dan ditampilkan kembali pada halaman konfirmasi tanpa disimpan ke database.

3.7 Mendefinisikan Route Sesuai Rancangan Tugas Akhir

Tugas kedua praktikum adalah membuat seluruh route sesuai rancangan Tugas Akhir dan memastikan setiap halaman dapat diakses melalui route. Pada rancangan Tugas Akhir saya terdapat proses registrasi data siswa yang terdiri dari halaman pengisian form dan halaman hasil data. Dua route berikut ditambahkan ke file routes/web.php:

```
Route::get('/formregistrasisiswa', [FormRegistrasiSiswaController::class, 'form'])->name('formregistrasisiswa.form');

Route::post('/formregistrasisiswa', [FormRegistrasiSiswaController::class, 'proses'])->name('formregistrasisiswa.proses');
```

```
Route::get('/formregistrasisiswa/konfirmasi', [FormRegistrasiSiswaController::class, 'konfirmasi'])-
>name('formregistrasisiswa.konfirmasi');
```

Route GET /formregistrasisiswa dapat dibuka langsung melalui browser untuk menampilkan halaman form registrasi, sedangkan route POST /formregistrasisiswa menerima data form dan menampilkan halaman hasil. Dengan adanya route ini, setiap halaman aplikasi registrasi siswa dapat diakses melalui URL yang jelas sesuai rancangan Tugas Akhir.

3.8 Membuat Aplikasi Input-Proses-Tampil Registrasi Siswa

Tugas ketiga adalah membuat aplikasi web sederhana tanpa database yang merepresentasikan salah satu proses pengolahan data (input, proses, dan tampilkan hasil) pada rancangan Tugas Akhir, yaitu proses registrasi data siswa. Controller dibuat dengan perintah artisan berikut:

```
php artisan make:controller FormRegistrasiSiswaController
```

Isi file app/Http/Controllers/FormRegistrasiSiswaController.php memvalidasi tujuh data masukan, yaitu nama lengkap, tanggal lahir, jenis kelamin, jenis disabilitas, kelas, nama orang tua, dan kontak orang tua. Validasi juga memakai aturan required_if untuk kolom keterangan disabilitas ketika pilihan jenis disabilitas bernilai Lainnya:

```
$validated = $request->validate([
    'nama_lengkap' => 'required|string|max:100',
    'tanggal_lahir' => 'required|date|before:today',
    'jenis_kelamin' => 'required|in:Laki-laki,Perempuan',
    'jenis_disabilitas' => 'required|string|max:100',
    'jenis_disabilitas_lainnya' => 'required_if:jenis_disabilitas,Lainnya|nullable|string|max:100',
    'kelas' => 'required|string|max:50',
    'nama_orang_tua' => 'required|string|max:100',
    'kontak_orang_tua' => 'required|string|min:8|max:30',
]);

$lahir = \Carbon\Carbon::parse($validated['tanggal_lahir']);
```

Method proses() kemudian menyusun array pendaftaran berisi nomor referensi, data siswa, usia yang dihitung dari tanggal lahir, data orang tua, dan waktu pendaftaran, lalu mengirimkannya ke halaman konfirmasi melalui session flash. Method konfirmasi() memeriksa session 'pendaftaran' dan mengembalikan pengguna ke halaman form jika session tidak ditemukan.

View form dibuat pada file resources/views/formregistrasisiswa/form.blade.php. Form memakai method POST ke route formregistrasisiswa.proses dengan directive @csrf. Jenis kelamin dan jenis disabilitas memakai elemen select, tanggal lahir memakai input type=date, dan kontak orang tua memakai input type=tel. Ketika pengguna memilih opsi Lainnya pada jenis disabilitas, JavaScript pada public/js/formregistrasisiswa.js menampilkan kolom isian tambahan. Inti dari form tersebut adalah sebagai berikut:

```
<select class="field__input" id="jenis_disabilitas" name="jenis_disabilitas" required>
    <option value="" disabled hidden>Pilih jenis disabilitas</option>
    <option value="Tunanetra">Tunanetra</option>
    <option value="Tunarungu">Tunarungu</option>
    <option value="Autisme">Autisme</option>
    <option value="Lainnya">Lainnya</option>
</select>

<div class="field" id="field-disabilitas-lainnya" hidden>
    <label class="field__label" for="jenis_disabilitas_lainnya">Sebutkan jenis disabilitas</label>
    <input class="field__input" type="text" id="jenis_disabilitas_lainnya"
           name="jenis_disabilitas_lainnya" disabled>
</div>
```

View hasil dibuat pada file resources/views/formregistrasisiswa/konfirmasi.blade.php. Data siswa ditampilkan dalam daftar baris label dan nilai memakai sintaks Blade {{ $variabel }}, dilengkapi badge jenis disabilitas dan tautan untuk mengisi form kembali. Pengujian dilakukan melalui php artisan serve: buka http://127.0.0.1:8000/formregistrasisiswa, isi ketujuh field, tekan tombol Kirim Pendaftaran, lalu halaman konfirmasi menampilkan kembali data yang dikirim beserta usia siswa.

4. Hasil dan Dokumentasi Program

Berikut adalah tangkapan layar kode program dan tampilan aplikasi LaporBanjir serta aplikasi Registrasi Siswa yang berjalan pada browser melalui php artisan serve. Bagian pertama mendokumentasikan jurnal praktikum berupa aplikasi LaporBanjir, sedangkan bagian kedua mendokumentasikan aplikasi Tugas Akhir berupa registrasi siswa.

4.1 Dokumentasi Jurnal: Aplikasi LaporBanjir

Potongan kode berikut ditampilkan dalam tampilan jendela editor agar mudah dibaca, lengkap dengan nomor baris dan pewarnaan sintaks.

routes/web.php

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\CobaController;
use App\Http\Controllers\DataController;
use App\Http\Controllers\MahasiswaController;
use App\Http\Controllers\LaporBanjirController;
use App\Http\Controllers\FormRegistrasiSiswaController;


Route::get('/', function () {
    return view('welcome');
});

Route::get('/home', function () {
    return 'Halo, ini adalah halaman Home';
})->name('home.page');

Route::get('/welcome', function () {
    return view ('welcome'); })->name('welcome.page');

Route::get('/coba', [CobaController::class, 'index']); {
};

Route::get('/final', [CobaController::class, 'final']); {
};

Route::get('/hello', function(){
    return 'Hello, Dryhus Dzacky Damingtyas.S';
});


Route::get('/user/{name}', function($name){
    return "Hello $name";
});

Route::get('/user/{name?}', function($name = 'Guest'){
    return "Hello $name";
});

Route::get('/about', function(){
    return view ('about', ['name' => 'Dryhus Dzacky Damingtyas.S']);
});

Route::get('/profile/{name?}', function($name = 'Guest'){
    return view ('profile', ['name' => $name]);
});

Route::get('/form', [DataController::class, 'form']); {
};

Route::post('/proses', [DataController::class, 'proses']); {
};

Route::get('/formMahasiswa', [MahasiswaController::class, 'form']); {
};

Route::post('/prosesMahasiswa', [MahasiswaController::class, 'proses']); {
};

Route::get('/laporbanjir', [LaporBanjirController::class, 'form'])->name('laporbanjir.form');

Route::post('/laporbanjir', [LaporBanjirController::class, 'proses'])->name('laporbanjir.proses');

Route::get('/laporbanjir/konfirmasi', [LaporBanjirController::class, 'konfirmasi'])->name('laporbanjir.konfirmasi');

Route::get('/formregistrasisiswa', [FormRegistrasiSiswaController::class, 'form'])->name('formregistrasisiswa.form');

Route::post('/formregistrasisiswa', [FormRegistrasiSiswaController::class, 'proses'])->name('formregistrasisiswa.proses');

Route::get('/formregistrasisiswa/konfirmasi', [FormRegistrasiSiswaController::class, 'konfirmasi'])->name('formregistrasisiswa.konfirmasi');
```

Gambar 1. Kode routes/web.php.

Gambar di atas menampilkan definisi route pada routes/web.php. Terlihat route latihan materi dan tiga route aplikasi LaporBanjir, yaitu GET /laporbanjir, POST /laporbanjir, dan GET /laporbanjir/konfirmasi, yang seluruhnya memakai named route.

app/Http/Controllers/LaporBanjirController.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class LaporBanjirController extends Controller
{
    public function form()
    {
        return view('laporbanjir.form');
    }

    public function proses(Request $request)
    {
        $validated = $request->validate([
            'nama' => 'required|string|max:100',
            'kecamatan' => 'required|string|max:100',
            'desa' => 'required|string|max:100',
            'tinggi_genangan' => 'required|numeric|min:0|max:1000',
        ], [
            'nama.required' => 'Nama pelapor wajib diisi.',
            'nama.max' => 'Nama pelapor maksimal 100 karakter.',
            'kecamatan.required' => 'Kecamatan lokasi kejadian wajib diisi.',
            'kecamatan.max' => 'Nama kecamatan maksimal 100 karakter.',
            'desa.required' => 'Desa atau kelurahan lokasi kejadian wajib diisi.',
            'desa.max' => 'Nama desa atau kelurahan maksimal 100 karakter.',
            'tinggi_genangan.required' => 'Tinggi genangan air wajib diisi.',
            'tinggi_genangan.numeric' => 'Tinggi genangan harus berupa angka.',
            'tinggi_genangan.min' => 'Tinggi genangan tidak boleh kurang dari 0.',
            'tinggi_genangan.max' => 'Tinggi genangan tidak masuk akal, periksa kembali pengukuran.',
        ]);

        $tinggi = (float) $validated['tinggi_genangan'];

        if ($tinggi >= 70) {
```

Gambar 2. Kode app/Http/Controllers/LaporBanjirController.php (bagian 1 dari 2).

app/Http/Controllers/LaporBanjirController.php

```
            $kategori = 'Banjir Tinggi';
            $deskripsiKategori = 'Air setinggi orang dewasa atau lebih, segera evakuasi ke tempat aman.';
        } elseif ($tinggi >= 30) {
            $kategori = 'Genangan Sedang';
            $deskripsiKategori = 'Air masuk permukiman dan bisa mengganggu lalu lintas warga.';
        } else {
            $kategori = 'Genangan Rendah';
            $deskripsiKategori = 'Air menggenang di permukaan jalan, masih bisa dilalui dengan hati-hati.';
        }

        $laporan = [
            'referensi' => 'LB-' . now()->format('Ymd-His'),
            'nama' => $validated['nama'],
            'kecamatan' => $validated['kecamatan'],
            'desa' => $validated['desa'],
            'tinggi' => $tinggi,
            'kategori' => $kategori,
            'deskripsiKategori' => $deskripsiKategori,
            'waktu' => now()->format('d-m-Y H:i'),
        ];

        return redirect()
            ->route('laporbanjir.konfirmasi')
            ->with('laporan', $laporan);
    }

    public function konfirmasi()
    {
        if (! session('laporan')) {
            return redirect()->route('laporbanjir.form');
        }

        return view('laporbanjir.konfirmasi', ['laporan' => session('laporan')]);
    }
}
```

Gambar 3. Kode app/Http/Controllers/LaporBanjirController.php (bagian 2 dari 2).

Gambar di atas menampilkan controller LaporBanjir yang berisi method form(), proses(), dan konfirmasi(). Method proses() memvalidasi masukan, menghitung kategori genangan, kemudian mengirim hasil olahan ke halaman konfirmasi melalui session flash.

resources/views/laporbanjir/form.blade.php

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>LaporBanjir | Formulir Laporan Banjir</title>
    <link rel="stylesheet" href="{{ asset('css/laporbanjir.css') }}">
</head>
<body>
    <header class="site-header">
        <div class="site-header__dalam">
            <p class="site-header__instansi">BPBD Kabupaten Bandung</p>
            <p class="site-header__brand">LaporBanjir</p>
        </div>
    </header>

    <main class="page">
        <h1 class="page__judul">Formulir Laporan Banjir</h1>
        <p class="page__intro">
            Laporkan kejadian banjir di wilayah Anda agar dapat ditindaklanjuti petugas BPBD.
            Isi data berikut, lalu tekan "Kirim Laporan". Prototipe ini hanya memproses dan
            menampilkan kembali data Anda, tidak menyimpannya.
        </p>

        <form class="kartu form" id="form-lapor" action="{{ route('laporbanjir.proses') }}" method="POST">
            @csrf

            <div class="field">
                <label class="field__label" for="nama">Nama pelapor</label>
                <input class="field__input @error('nama') field__input--galat @enderror"
                       type="text" id="nama" name="nama"
                       value="{{ old('nama') }}" maxlength="100"
                       autocomplete="name" required>
                @error('nama')
                    <p class="field__galat" role="alert">{{ $message }}</p>
                @enderror
            </div>

            <div class="field">
                <label class="field__label" for="kecamatan">Kecamatan lokasi kejadian</label>
                <input class="field__input @error('kecamatan') field__input--galat @enderror"
                       type="text" id="kecamatan" name="kecamatan"
                       value="{{ old('kecamatan') }}" maxlength="100"
                       placeholder="Contoh: Soreang" required>
```

Gambar 4. Kode resources/views/laporbanjir/form.blade.php (bagian 1 dari 2).

resources/views/laporbanjir/form.blade.php

```
                @error('kecamatan')
                    <p class="field__galat" role="alert">{{ $message }}</p>
                @enderror
            </div>

            <div class="field">
                <label class="field__label" for="desa">Desa atau kelurahan</label>
                <input class="field__input @error('desa') field__input--galat @enderror"
                       type="text" id="desa" name="desa"
                       value="{{ old('desa') }}" maxlength="100"
                       placeholder="Nama desa atau kelurahan" required>
                @error('desa')
                    <p class="field__galat" role="alert">{{ $message }}</p>
                @enderror
            </div>

            <div class="field">
                <label class="field__label" for="tinggi_genangan">Tinggi genangan air (cm)</label>
                <div class="field__satuan">
                    <input class="field__input field__input--angka @error('tinggi_genangan') field__input--galat @enderror"
                           type="number" id="tinggi_genangan" name="tinggi_genangan"
                           value="{{ old('tinggi_genangan') }}" min="0" max="1000" step="1"
                           inputmode="numeric" required>
                    <span class="field__unit" aria-hidden="true">cm</span>
                </div>
                <p class="field__hint">Perkiraan tinggi air di lokasi kejadian. Contoh: 40.</p>
                <p class="field__hint field__hint--live" id="preview-kategori" hidden></p>
                @error('tinggi_genangan')
                    <p class="field__galat" role="alert">{{ $message }}</p>
                @enderror
            </div>

            <button class="tombol" type="submit" id="tombol-kirim">Kirim Laporan</button>
        </form>
    </main>

    <footer class="site-footer">
        <p>Prototipe LaporBanjir untuk praktikum PABW. Data laporan tidak disimpan secara permanen.</p>
    </footer>

    <script src="{{ asset('js/laporbanjir.js') }}"></script>
</body>
</html>
```

Gambar 5. Kode resources/views/laporbanjir/form.blade.php (bagian 2 dari 2).

Gambar di atas menampilkan view form pelaporan yang memuat CSS eksternal, directive @csrf, input nama pelapor, kecamatan, desa, dan tinggi genangan, serta penanganan pesan galat menggunakan directive @error.

resources/views/laporbanjir/konfirmasi.blade.php

```blade
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>LaporBanjir | Konfirmasi Laporan</title>
    <link rel="stylesheet" href="{{ asset('css/laporbanjir.css') }}">
</head>
@php
    $tinggiTampil = rtrim(rtrim(number_format($laporan['tinggi'], 1, ',', '.'), '0'), ',');
    $persen = max(0, min(100, $laporan['tinggi']));
    $posisi = $persen < 12 ? 'kiri' : ($persen > 88 ? 'kanan' : 'tengah');
    $kelasKategori = str_contains($laporan['kategori'], 'Tinggi')
        ? 'badge--tinggi'
        : (str_contains($laporan['kategori'], 'Sedang') ? 'badge--sedang' : 'badge--rendah');
@endphp
<body>
    <header class="site-header">
        <div class="site-header__dalam">
            <p class="site-header__instansi">BPBD Kabupaten Bandung</p>
            <p class="site-header__brand">LaporBanjir</p>
        </div>
    </header>

    <main class="page">
        <p class="status status--ok" role="status">Laporan berhasil diproses.</p>
        <h1 class="page__judul">Konfirmasi Laporan</h1>
        <p class="page__intro">
            Terima kasih, {{ $laporan['nama'] }}. Berikut ringkasan laporan yang Anda kirimkan.
        </p>

        <div class="kartu ringkasan">
            <dl class="ringkasan__daftar">
                <div class="ringkasan__baris">
                    <dt>Nomor referensi</dt>
                    <dd>{{ $laporan['referensi'] }}</dd>
                </div>
                <div class="ringkasan__baris">
                    <dt>Nama pelapor</dt>
                    <dd>{{ $laporan['nama'] }}</dd>
                </div>
                <div class="ringkasan__baris">
                    <dt>Lokasi kejadian</dt>
                    <dd>Desa {{ $laporan['desa'] }}, Kecamatan {{ $laporan['kecamatan'] }}</dd>
                </div>
                <div class="ringkasan__baris">
                    <dt>Waktu lapor</dt>
```

Gambar 6. Kode resources/views/laporbanjir/konfirmasi.blade.php (bagian 1 dari 2).

resources/views/laporbanjir/konfirmasi.blade.php

```
                    <dd>{{ $laporan['waktu'] }}</dd>
                </div>
            </dl>

            <div class="genangan">
                <div class="genangan__kepala">
                    <h2 class="genangan__judul">Tinggi genangan</h2>
                    <p class="genangan__nilai">{{ $tinggiTampil }} cm</p>
                </div>

                <div class="gauge" role="img"
                     aria-label="Skala tinggi genangan {{ $tinggiTampil }} sentimeter, kategori {{ $laporan['kategori'] }}">
                    <div class="gauge__batang">
                        <span class="gauge__zona gauge__zona--rendah"></span>
                        <span class="gauge__zona gauge__zona--sedang"></span>
                        <span class="gauge__zona gauge__zona--tinggi"></span>
                        <span class="gauge__penunjuk gauge__penunjuk--{{ $posisi }}" style="left: {{ $persen }}%;">
                            <span class="gauge__label">{{ $tinggiTampil }} cm</span>
                        </span>
                    </div>
                    <div class="gauge__skala" aria-hidden="true">
                        <span>0</span>
                        <span>30</span>
                        <span>70</span>
                        <span>100+</span>
                    </div>
                </div>

                <p class="badge {{ $kelasKategori }}">{{ $laporan['kategori'] }}</p>
                <p class="genangan__ket">{{ $laporan['deskripsiKategori'] }}</p>
                <p class="genangan__sumber">Kategori dihitung otomatis dari tinggi genangan yang Anda laporkan.</p>
            </div>
        </div>

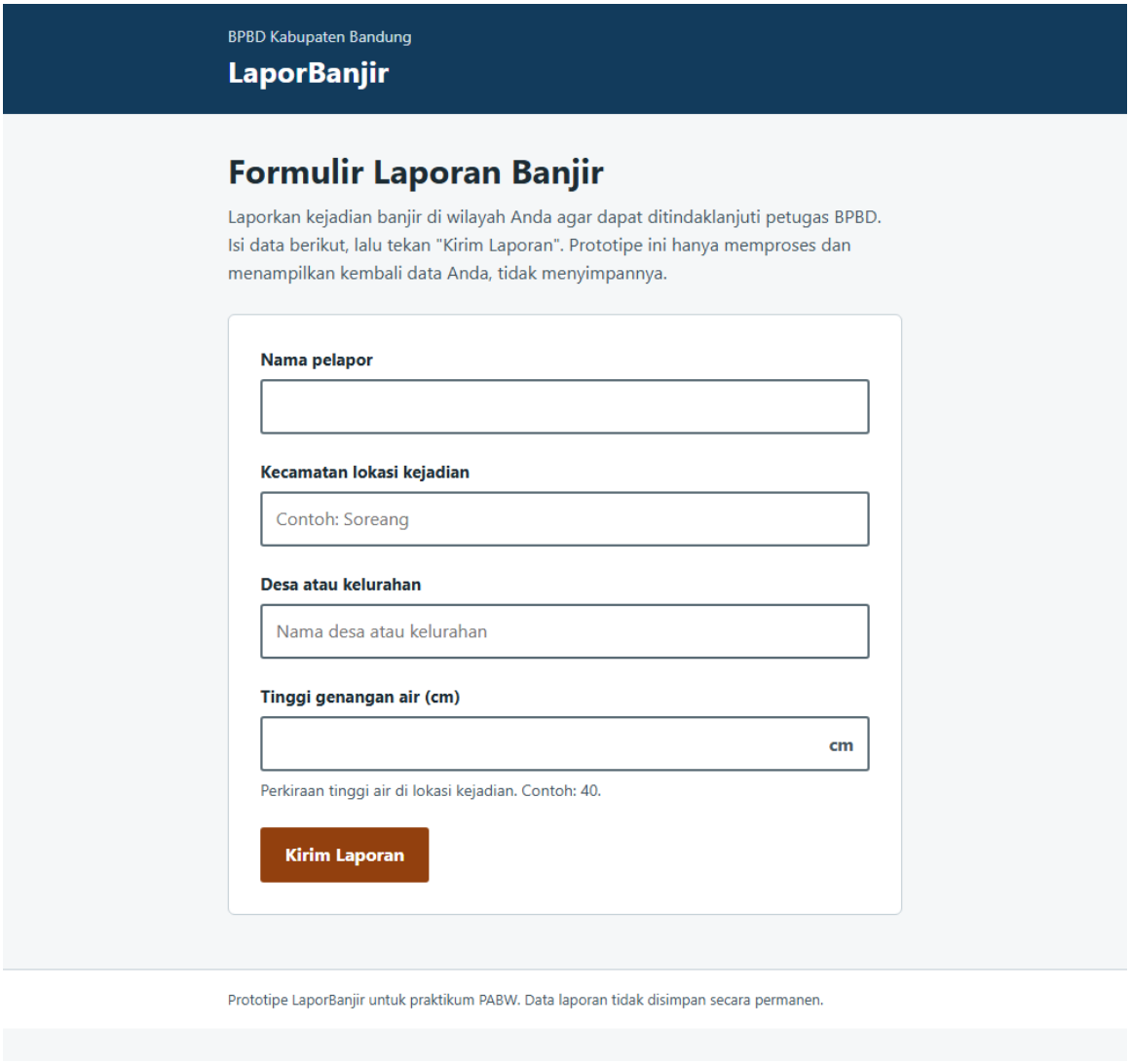
        <p class="catatan">
            Catatan: prototipe ini tidak menyimpan laporan. Data di halaman ini hilang
            setelah Anda meninggalkannya.
        </p>

        <a class="tombol tombol--link" href="{{ route('laporbanjir.form') }}">Buat laporan lain</a>
    </main>

    <footer class="site-footer">
        <p>Prototipe LaporBanjir untuk praktikum PABW. Data laporan tidak disimpan secara permanen.</p>
    </footer>
</body>
</html>
```

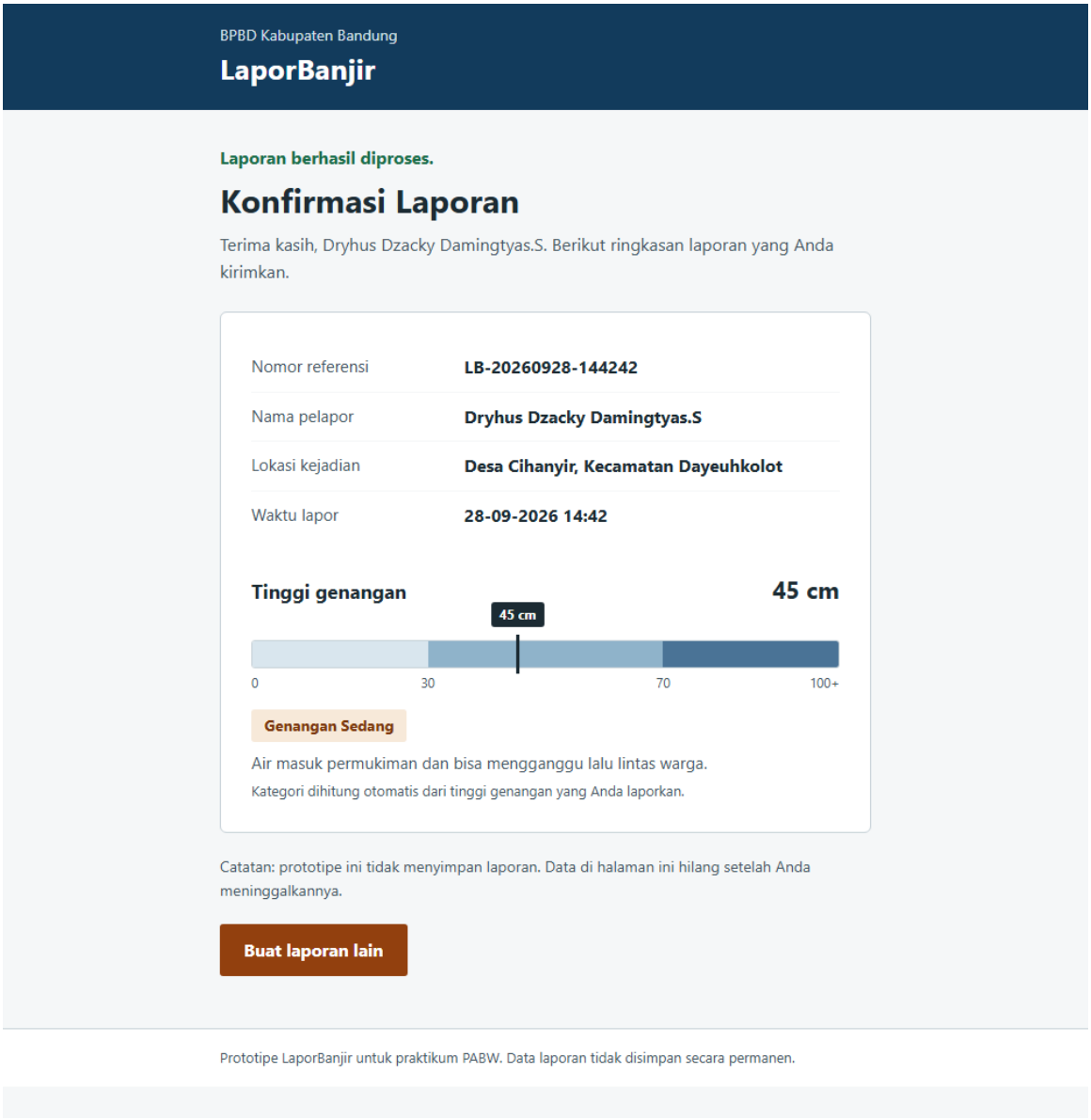
Gambar 7. Kode resources/views/laporbanjir/konfirmasi.blade.php (bagian 2 dari 2).

Gambar di atas menampilkan view konfirmasi yang menyusun ringkasan laporan dan visualisasi tinggi genangan beserta badge kategori yang dihitung pada controller.



Gambar 8. Halaman form pelaporan banjir pada alamat /laporbanjir.

Gambar di atas menampilkan halaman awal aplikasi berupa form pelaporan yang berisi input nama pelapor, kecamatan, desa, dan tinggi genangan air dalam satuan sentimeter, serta tombol Kirim Laporan. Form ditampilkan dalam kartu putih dengan header biru dan tata letak terpusat agar mudah digunakan.



Gambar 9. Halaman konfirmasi laporan pada /laporbanjir/konfirmasi setelah form dikirim.

Gambar di atas menampilkan halaman hasil setelah form diisi dan tombol Kirim Laporan ditekan. Data yang dikirim melalui method POST, yaitu nama pelapor (Dryhus Dzacky Damingtyas.S), lokasi kejadian (Kecamatan Dayeuhkolot, Desa Cihanyir), dan tinggi genangan (45 cm), berhasil diterima controller dan ditampilkan kembali sebagai konfirmasi laporan. Tinggi 45 cm termasuk kategori Genangan Sedang, dan data tidak disimpan secara permanen sesuai konsep prototipe tanpa database.

4.2 Dokumentasi Tugas Akhir: Aplikasi Registrasi Siswa

app/Http/Controllers/FormRegistrasiSiswaController.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class FormRegistrasiSiswaController extends Controller
{
    public function form()
    {
        return view('formregistrasisiswa.form');
    }

    public function proses(Request $request)
    {
        $validated = $request->validate([
            'nama_lengkap' => 'required|string|max:100',
            'tanggal_lahir' => 'required|date|before:today',
            'jenis_kelamin' => 'required|in:Laki-laki,Perempuan',
            'jenis_disabilitas' => 'required|string|max:100',
            'jenis_disabilitas_lainnya' => 'required_if:jenis_disabilitas,Lainnya|nullable|string|max:100',
            'kelas' => 'required|string|max:50',
            'nama_orang_tua' => 'required|string|max:100',
            'kontak_orang_tua' => 'required|string|min:8|max:30',
        ], [
            'nama_lengkap.required' => 'Nama lengkap wajib diisi.',
            'nama_lengkap.max' => 'Nama lengkap maksimal 100 karakter.',
            'tanggal_lahir.required' => 'Tanggal lahir wajib diisi.',
            'tanggal_lahir.date' => 'Tanggal lahir harus berupa tanggal yang valid.',
            'tanggal_lahir.before' => 'Tanggal lahir harus sebelum hari ini.',
            'jenis_kelamin.required' => 'Jenis kelamin wajib dipilih.',
            'jenis_kelamin.in' => 'Pilihan jenis kelamin tidak sesuai.',
            'jenis_disabilitas.required' => 'Jenis disabilitas wajib dipilih.',
            'jenis_disabilitas_lainnya.required_if' => 'Sebutkan jenis disabilitas pada kolom isian.',
            'jenis_disabilitas_lainnya.max' => 'Keterangan disabilitas maksimal 100 karakter.',
            'kelas.required' => 'Kelas wajib diisi.',
            'kelas.max' => 'Kelas maksimal 50 karakter.',
            'nama_orang_tua.required' => 'Nama orang tua wajib diisi.',
            'nama_orang_tua.max' => 'Nama orang tua maksimal 100 karakter.',
```

Gambar 10. Kode app/Http/Controllers/FormRegistrasiSiswaController.php (bagian 1 dari 2).

app/Http/Controllers/FormRegistrasiSiswaController.php

```
            'kontak_orang_tua.required' => 'Kontak orang tua wajib diisi.',
            'kontak_orang_tua.min' => 'Kontak orang tua minimal 8 karakter.',
            'kontak_orang_tua.max' => 'Kontak orang tua maksimal 30 karakter.',
        ]);

        $disabilitas = $validated['jenis_disabilitas'] === 'Lainnya'
            ? $validated['jenis_disabilitas_lainnya']
            : $validated['jenis_disabilitas'];

        $lahir = \Carbon\Carbon::parse($validated['tanggal_lahir']);

        $pendaftaran = [
            'referensi' => 'REG-' . now()->format('Ymd-His'),
            'nama' => $validated['nama_lengkap'],
            'tanggal_lahir' => $lahir->format('d-m-Y'),
            'usia' => $lahir->age,
            'jenis_kelamin' => $validated['jenis_kelamin'],
            'disabilitas' => $disabilitas,
            'kelas' => $validated['kelas'],
            'orang_tua' => $validated['nama_orang_tua'],
            'kontak' => $validated['kontak_orang_tua'],
            'waktu' => now()->format('d-m-Y H:i'),
        ];

        return redirect()
            ->route('formregistrasisiswa.konfirmasi')
            ->with('pendaftaran', $pendaftaran);
    }

    public function konfirmasi()
    {
        if (! session('pendaftaran')) {
            return redirect()->route('formregistrasisiswa.form');
        }

        return view('formregistrasisiswa.konfirmasi', ['pendaftaran' => session('pendaftaran')]);
    }
}
```

Gambar 11. Kode app/Http/Controllers/FormRegistrasiSiswaController.php (bagian 2 dari 2).

Gambar di atas menampilkan controller registrasi siswa yang memvalidasi tujuh data masukan, menghitung usia dari tanggal lahir, dan menyusun array pendaftaran yang dikirim ke halaman konfirmasi melalui session flash.

resources/views/formregistrasisiswa/form.blade.php

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registrasi Siswa | Formulir Pendaftaran</title>
    <link rel="stylesheet" href="{{ asset('css/formregistrasisiswa.css') }}">
</head>
<body>
    <header class="site-header">
        <div class="site-header__dalam">
            <p class="site-header__instansi">Sekolah Inklusif</p>
            <p class="site-header__brand">Registrasi Siswa</p>
        </div>
    </header>

    <main class="page">
        <h1 class="page__judul">Formulir Pendaftaran Siswa</h1>
        <p class="page__intro">
            Isi data calon siswa dengan benar agar sekolah dapat menyiapkan pendampingan
            yang sesuai. Tekan "Kirim Pendaftaran" untuk melihat ringkasan. Prototipe ini
            hanya memproses dan menampilkan kembali data Anda, tidak menyimpannya.
        </p>

        <form class="kartu form" id="form-daftar" action="{{ route('formregistrasisiswa.proses') }}" method="POST">
            @csrf

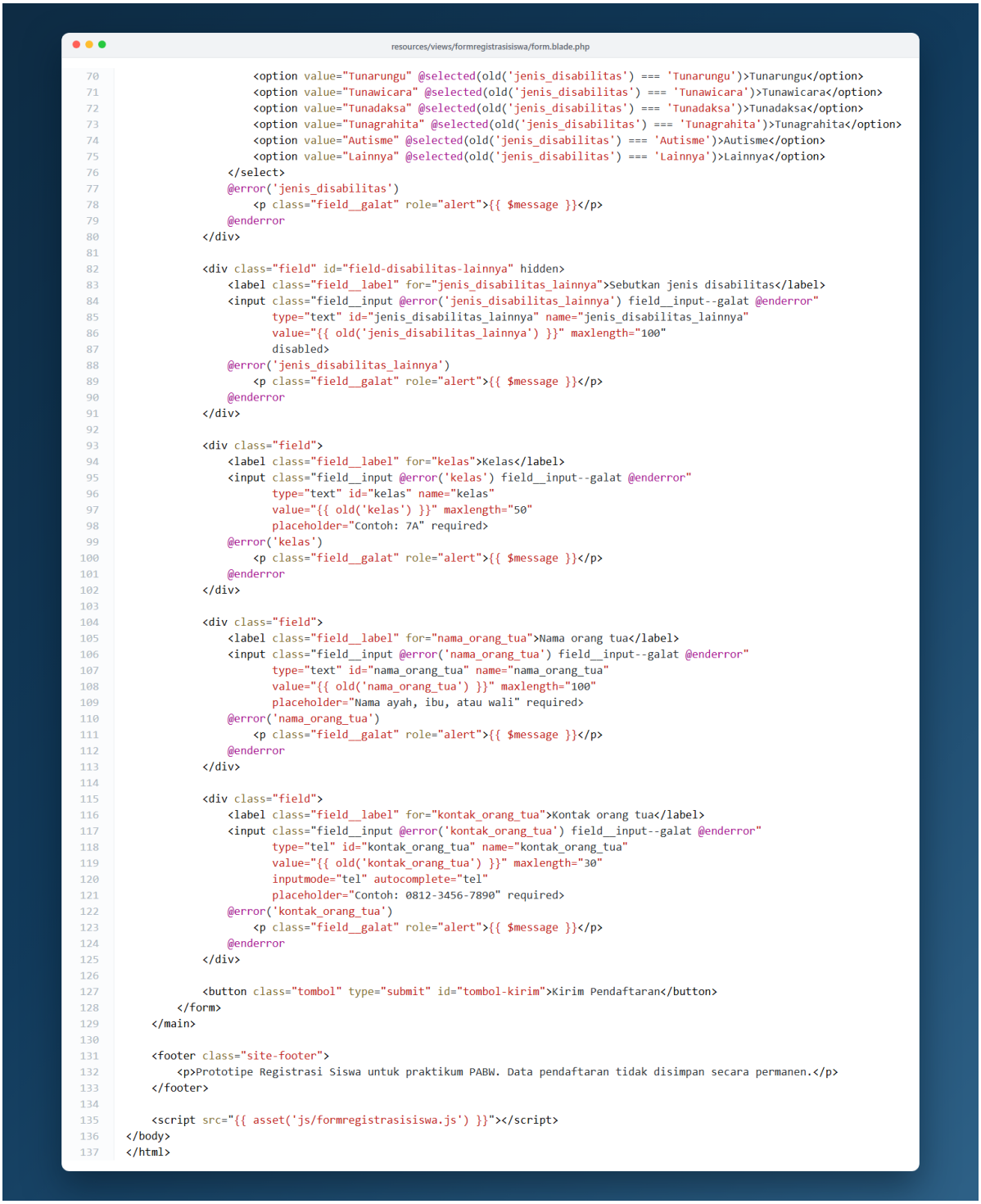
            <div class="field">
                <label class="field__label" for="nama_lengkap">Nama lengkap</label>
                <input class="field__input @error('nama_lengkap') field__input--galat @enderror"
                       type="text" id="nama_lengkap" name="nama_lengkap"
                       value="{{ old('nama_lengkap') }}" maxlength="100"
                       autocomplete="name" required>
                @error('nama_lengkap')
                    <p class="field__galat" role="alert">{{ $message }}</p>
                @enderror
            </div>

            <div class="field">
                <label class="field__label" for="tanggal_lahir">Tanggal lahir</label>
                <input class="field__input @error('tanggal_lahir') field__input--galat @enderror"
                       type="date" id="tanggal_lahir" name="tanggal_lahir"
                       value="{{ old('tanggal_lahir') }}"
                       max="{{ now()->format('Y-m-d') }}" required>
                <p class="field__hint field__hint--live" id="preview-usia" hidden></p>
                @error('tanggal_lahir')
                    <p class="field__galat" role="alert">{{ $message }}</p>
                @enderror
            </div>

            <div class="field">
                <label class="field__label" for="jenis_kelamin">Jenis kelamin</label>
                <select class="field__input @error('jenis_kelamin') field__input--galat @enderror"
                        id="jenis_kelamin" name="jenis_kelamin" required>
                    <option value="" disabled @selected(old('jenis_kelamin') === null) hidden>Pilih jenis kelamin</option>
                    <option value="Laki-laki" @selected(old('jenis_kelamin') === 'Laki-laki')>Laki-laki</option>
                    <option value="Perempuan" @selected(old('jenis_kelamin') === 'Perempuan')>Perempuan</option>
                </select>
                @error('jenis_kelamin')
                    <p class="field__galat" role="alert">{{ $message }}</p>
                @enderror
            </div>

            <div class="field">
                <label class="field__label" for="jenis_disabilitas">Jenis disabilitas</label>
                <select class="field__input @error('jenis_disabilitas') field__input--galat @enderror"
                        id="jenis_disabilitas" name="jenis_disabilitas" required>
                    <option value="" disabled @selected(old('jenis_disabilitas') === null) hidden>Pilih jenis disabilitas</option>
                    <option value="Tunanetra" @selected(old('jenis_disabilitas') === 'Tunanetra')>Tunanetra</option>
```

Gambar 12. Kode resources/views/formregistrasisiswa/form.blade.php (bagian 1 dari 2).



Gambar 13. Kode resources/views/formregistrasisiswa/form.blade.php (bagian 2 dari 2).

Gambar di atas menampilkan view form registrasi siswa yang memuat tujuh field, directive @csrf, elemen select untuk jenis kelamin dan jenis disabilitas, serta kolom isian kondisional untuk opsi Lainnya.

resources/views/formregistrasisiswa/konfirmasi.blade.php

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registrasi Siswa | Konfirmasi Pendaftaran</title>
    <link rel="stylesheet" href="{{ asset('css/formregistrasisiswa.css') }}">
</head>
<body>
    <header class="site-header">
        <div class="site-header__dalam">
            <p class="site-header__instansi">Sekolah Inklusif</p>
            <p class="site-header__brand">Registrasi Siswa</p>
        </div>
    </header>

    <main class="page">
        <p class="status status--ok" role="status">Pendaftaran berhasil diproses.</p>
        <h1 class="page__judul">Konfirmasi Pendaftaran</h1>
        <p class="page__intro">
            Terima kasih, {{ $pendaftaran['nama'] }}. Berikut ringkasan data pendaftaran
            yang Anda kirimkan.
        </p>

        <div class="kartu ringkasan">
            <dl class="ringkasan__daftar">
                <div class="ringkasan__baris">
                    <dt>Nomor referensi</dt>
                    <dd>{{ $pendaftaran['referensi'] }}</dd>
                </div>
                <div class="ringkasan__baris">
                    <dt>Nama lengkap</dt>
                    <dd>{{ $pendaftaran['nama'] }}</dd>
                </div>
                <div class="ringkasan__baris">
                    <dt>Tanggal lahir</dt>
                    <dd>{{ $pendaftaran['tanggal_lahir'] }} ({{ $pendaftaran['usia'] }} tahun)</dd>
                </div>
                <div class="ringkasan__baris">
                    <dt>Jenis kelamin</dt>
                    <dd>{{ $pendaftaran['jenis_kelamin'] }}</dd>
                </div>
                <div class="ringkasan__baris">
```

Gambar 14. Kode resources/views/formregistrasisiswa/konfirmasi.blade.php (bagian 1 dari 2).

resources/views/formregistrasisiswa/konfirmasi.blade.php

```
                    <dt>Kelas</dt>
                    <dd>{{ $pendaftaran['kelas'] }}</dd>
                </div>
                <div class="ringkasan__baris">
                    <dt>Nama orang tua</dt>
                    <dd>{{ $pendaftaran['orang_tua'] }}</dd>
                </div>
                <div class="ringkasan__baris">
                    <dt>Kontak orang tua</dt>
                    <dd>{{ $pendaftaran['kontak'] }}</dd>
                </div>
                <div class="ringkasan__baris">
                    <dt>Waktu pendaftaran</dt>
                    <dd>{{ $pendaftaran['waktu'] }}</dd>
                </div>
            </dl>

            <div class="dukungan">
                <div class="dukungan__kepala">
                    <h2 class="dukungan__judul">Jenis disabilitas</h2>
                </div>
                <p class="badge badge--netral">{{ $pendaftaran['disabilitas'] }}</p>
                <p class="dukungan__ket">
                    Data ini membantu sekolah menyiapkan pendampingan belajar yang sesuai
                    bagi calon siswa.
                </p>
            </div>
        </div>

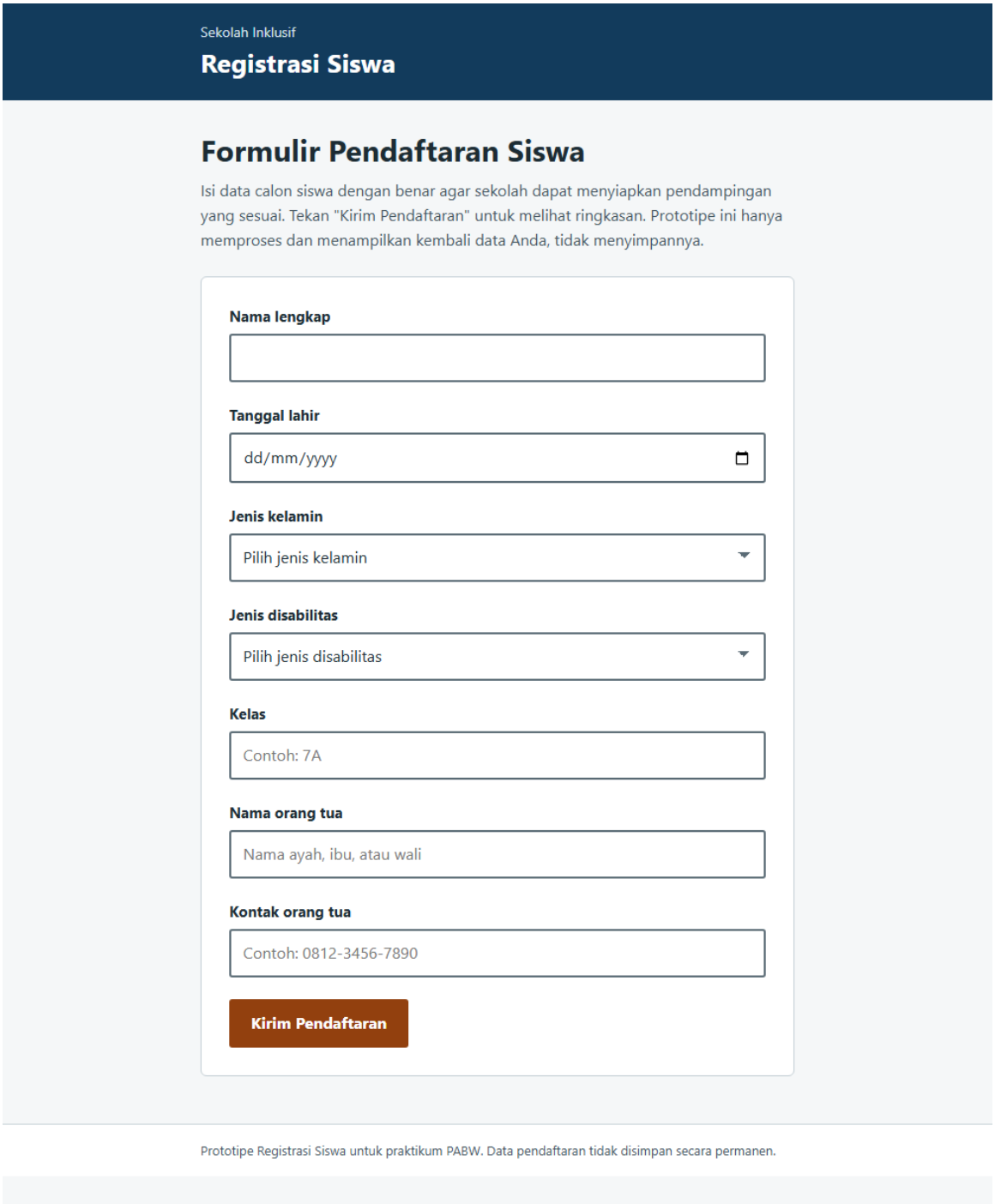
        <p class="catatan">
            Catatan: prototipe ini tidak menyimpan data. Isi halaman ini hilang setelah
            Anda meninggalkannya.
        </p>

        <a class="tombol tombol--link" href="{{ route('formregistrasisiswa.form') }}">Daftarkan siswa lain</a>
    </main>

    <footer class="site-footer">
        <p>Prototipe Registrasi Siswa untuk praktikum PABW. Data pendaftaran tidak disimpan secara permanen.</p>
    </footer>
</body>
</html>
```

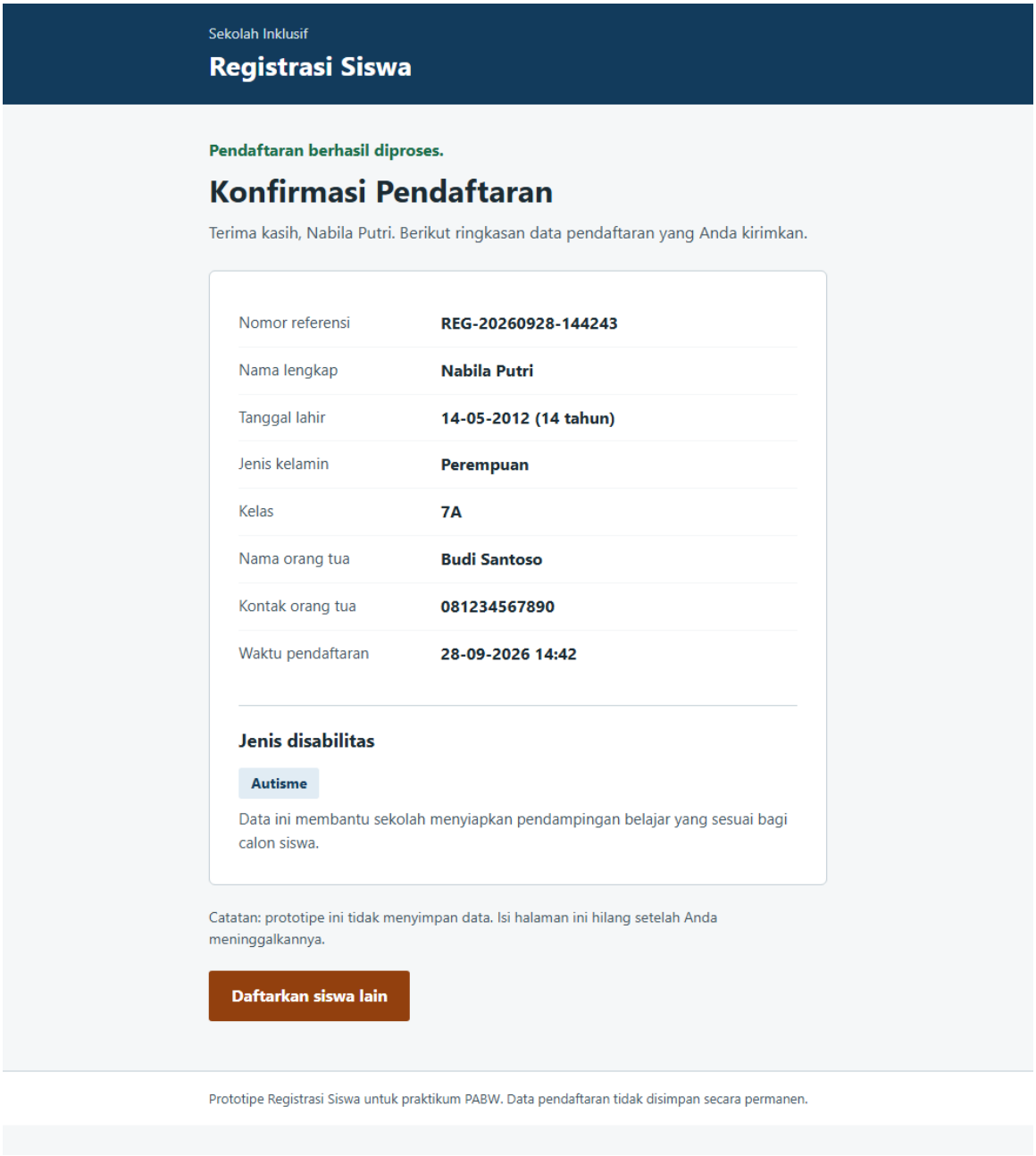
Gambar 15. Kode resources/views/formregistrasisiswa/konfirmasi.blade.php (bagian 2 dari 2).

Gambar di atas menampilkan view konfirmasi registrasi siswa yang menyusun ringkasan data siswa, data orang tua, dan badge jenis disabilitas.



Gambar 16. Halaman form pendaftaran siswa pada alamat /formregistrasisiswa.

Gambar di atas menampilkan form pendaftaran siswa yang terdiri dari tujuh field, yaitu nama lengkap, tanggal lahir (input date), jenis kelamin (select), jenis disabilitas (select dengan opsi Lainnya), kelas, nama orang tua, dan kontak orang tua, serta tombol Kirim Pendaftaran.



Gambar 17. Halaman konfirmasi pendaftaran siswa pada /formregistrasisiswa/konfirmasi setelah form dikirim.

Gambar di atas menampilkan halaman hasil setelah form registrasi dikirim. Data contoh yang diterima controller, yaitu Nabila Putri, lahir 14-05-2012, jenis kelamin Perempuan, jenis disabilitas Autisme, kelas 7A, orang tua Budi Santoso dengan kontak 081234567890, ditampilkan kembali dalam ringkasan beserta usia siswa dan badge jenis disabilitas. Data hanya ditampilkan sebagai hasil olahan tanpa disimpan ke database, sesuai ketentuan praktikum tanpa database.

5. Analisis dan Pembahasan

5.1 Pemisahan Route GET dan POST

Struktur route pada aplikasi LaporBanjir dan Registrasi Siswa sengaja dipisah menjadi dua, yaitu route GET yang hanya menampilkan halaman input dan route POST yang menerima data hasil pengiriman form. Pemisahan ini sesuai kaidah HTTP, di mana GET dipakai untuk menampilkan data

dan POST dipakai untuk mengirim atau mengubah data. Karena form memakai method POST, route tujuannya juga harus Route::post(); jika form dikirim ke route GET maka Laravel akan menolaknya dengan error MethodNotAllowedHttpException.

5.2 Peran Controller dan Pola MVC

Controller dipilih sebagai tempat logika pemrosesan agar route tetap ringkas dan sesuai pola MVC. Method proses() membaca data dari objek Request melalui $request->validate(), lalu mengirimkan hasil olahan ke view menggunakan session flash. Pada praktikum ini hasil tidak langsung dikembalikan dengan return view(), melainkan melalui redirect()->route(...)->with(...). Pola ini menjaga prinsip Post/Redirect/Get, sehingga ketika halaman dikonfirmasi disegarkan kembali, browser tidak mengirim ulang data form dan tidak memunculkan dialog pengiriman ulang. Method konfirmasi() juga memeriksa session sebelum menampilkan halaman, sehingga halaman konfirmasi tidak dapat dibuka langsung tanpa data kiriman.

5.3 Keamanan CSRF dan Validasi Server-Side

Setiap form menyertakan directive @csrf agar token keamanan ikut terkirim dan request POST dikenali sebagai request yang sah. Selain validasi di sisi browser melalui atribut required dan maxlength, controller juga melakukan validasi di sisi server melalui $request->validate() dengan pesan galat berbahasa Indonesia. Pesan galat tersebut ditampilkan kembali pada masing-masing field melalui directive @error dan nilai yang sudah diisi dikembalikan melalui helper old(), sehingga pengguna tidak perlu mengetik ulang seluruh data ketika terjadi kesalahan.

5.4 Logika Kategori dan Perhitungan Usia

Pada aplikasi LaporBanjir, kategori genangan dihitung dari tinggi air memakai percabangan if, elseif, dan else, yaitu 70 cm atau lebih termasuk Banjir Tinggi, 30 sampai 69 cm termasuk Genangan Sedang, dan di bawah 30 cm termasuk Genangan Rendah. Pada aplikasi Registrasi Siswa, usia dihitung dari tanggal lahir memakai Carbon sehingga halaman konfirmasi menampilkan usia siswa secara otomatis. Pemilihan input type=date dan input type=tel membantu browser menampilkan kontrol isian yang sesuai, sedangkan batas max pada input tanggal mencegah pengguna mengisi tanggal lahir di masa depan.

5.5 Field Kondisional Jenis Disabilitas

Jenis disabilitas disediakan sebagai daftar pilihan agar pengisian lebih konsisten, tetapi tetap disediakan opsi Lainnya untuk kondisi yang tidak tercakup. Ketika opsi Lainnya dipilih, JavaScript menampilkan kolom isian tambahan dan mengaktifkannya, sedangkan pada saat proses validasi server memakai aturan required_if sehingga kolom keterangan wajib diisi hanya jika opsi Lainnya yang dipilih. Nilai yang dikirim ke halaman konfirmasi adalah keterangan yang diisi pengguna, bukan kata Lainnya. Pendekatan ini menjaga data tetap jelas tanpa membatasi pilihan pada daftar yang tersedia.

5.6 Pemisahan CSS dan JavaScript Eksternal

Berbeda dengan CSS inline, styling dan perilaku halaman pada praktikum ini dipisahkan ke berkas eksternal pada public/css dan public/js, lalu dimuat melalui helper asset(). Cara ini membuat file Blade tetap fokus pada struktur dan logika tampilan, mengurangi pengulangan karena beberapa halaman memakai berkas gaya yang sama, serta memudahkan perubahan tampilan pada satu tempat. JavaScript yang dipakai hanya sebagai peningkatan pengalaman, misalnya menonaktifkan tombol saat pengiriman berlangsung dan memunculkan pratinjau kategori, sehingga aplikasi tetap berfungsi ketika JavaScript dimatikan.

5.7 Kendala dan Penyelesaian

1. Error 419 Page Expired saat mengirim form. Pengiriman data melalui method POST ditolak karena Laravel mensyaratkan token CSRF pada setiap permintaan POST. Masalah diselesaikan dengan menambahkan directive @csrf di dalam tag form.

2. Halaman konfirmasi tidak menampilkan data. Data hilang karena dikirim ke view tanpa melalui session flash. Masalah diselesaikan dengan memakai redirect()->route(...)->with('laporan', $laporan) dan memeriksa session pada method konfirmasi().

3. Pesan galat validasi tidak muncul. Pesan hanya tampil jika blok directive @error ditempatkan tepat di bawah setiap field, sehingga setiap input diberi blok @error masing-masing.

4. Opsi Lainnya pada jenis disabilitas tetap mengirim nilai kosong. Masalah diselesaikan dengan mengaktifkan atribut required pada input keterangan melalui JavaScript serta aturan validasi required_if pada controller.

6. Kesimpulan

1. Praktikum Modul 2 ini memperlihatkan struktur direktori proyek Laravel serta peran folder app, routes, resources, dan public dalam pola arsitektur MVC. Route meneruskan request, controller mengolah data, dan view menampilkan hasil.

2. Routing pada Laravel dapat dibuat dalam berbagai variasi, mulai dari route sederhana, route dengan parameter dan parameter opsional, route menuju view, pengiriman data ke view, hingga named route. Named route memudahkan pemanggilan alamat halaman menjadi lebih konsisten.

3. Aplikasi LaporBanjir berhasil dibangun tanpa database dengan alur yang utuh, yaitu pengguna membuka halaman form melalui route GET, mengirim data melalui method POST yang dilindungi token CSRF, kemudian controller memproses data dan menampilkannya kembali pada halaman konfirmasi beserta kategori genangan yang dihitung otomatis.

4. Aplikasi Registrasi Siswa berhasil dibangun sebagai representasi proses pengolahan data pada rancangan Tugas Akhir dengan alur input, proses, dan tampilkan hasil. Ketujuh data masukan berhasil divalidasi dan ditampilkan kembali pada halaman konfirmasi, termasuk penanganan opsi jenis disabilitas Lainnya melalui field kondisional.

5. Pemisahan CSS dan JavaScript ke berkas eksternal pada folder public membuat file view lebih ringkas dan mudah dirawat, sedangkan penggunaan session flash pada proses pengalihan menjaga pola Post/Redirect/Get sehingga aplikasi berjalan sesuai rancangan tanpa penyimpanan permanen.

7. Link Repository Github

[https://github.com/Dryhus21/PABW-PRAKTIKUM]

Repositori di atas memuat kode program lengkap aplikasi LaporBanjir dan aplikasi Registrasi Siswa beserta seluruh route, controller, dan view yang digunakan pada praktikum ini.